InfraRSI: Harnessing Recursive Self-Improvement for Training Infrastructure

"Command Once, and Self-Evolve: Your Personal Infra Agent for All Environments."

InfraRSI Team

arXiv:2611.11558v1 [cs.AI] 8 Nov 2026

Abstract

While Recursive Self-Improvement (RSI) has accelerated isolated algorithm and kernel synthesis, distributed LLM training throughput depends on tightly coupled launch settings, operator kernels, and communication schedules, where single edits often hit out-of-memory (OOM) boundaries, compiler conflicts, measurement jitter, or silent numerical drift. We present **InfraRSI**, the agentic harness that turns open-ended human commands into verified training-system optimizations through a five-stage workflow: `init` locks editable scope and a fixed task contract over workload, loss, and timing invariants to prevent reward hacking, `doctor` profiles baseline phases and calibrates cluster noise, `run` co-optimizes launch configurations and code patches before verifying top candidates in back-to-back baseline re-runs and writing checked findings to a cross-hardware experience bank, and `report` and `apply` audit source paths, account for failed-trial costs, and commit promoted changes. Rather than stepping greedily from a single configuration during `run`, InfraRSI initializes from diverse experience-seeded configurations and maintains global execution-time and censored peak-memory models fitted to all trials, including OOM failures. Using these models, it estimates per-setting memory costs, revises miscalibrated LLM speed priors from live measurements, and pairs memory-hungry optimizations with memory-saving settings and verified code patches. Across four single- and multi-node dense and Mixture-of-Experts (MoE) LLM training workloads, InfraRSI achieves a speedup of up to 2.75× and lifts mean Model FLOPs Utilization (MFU) from 17.4% to 30.0%, outperforming existing configuration tuners and coding agents while transferring experience across GPU architectures. As it improves training infrastructure efficiency, InfraRSI forms a two-level Recursive Self-Improvement loop—revising priors online within runs and distilling experience across clusters—to push foundation model scaling.

GitHub: https://github.com/TokenRhythm/InfraRSI
Project Page: https://infrarsi.github.io

1. Introduction

Building on classic formulations of self-referential improvement [34], recent Recursive Self-Improvement (RSI) systems demonstrate that large language models (LLMs) can discover mathematical constructions [33, 25], synthesize machine learning pipelines [14], and rewrite agentic scaffolding [45]. While these systems target isolated algorithms or single kernels, compute-optimal scaling laws [12] increasingly bind foundation model progress to the throughput of distributed training infrastructure [36, 24, 30, 29, 46]: launch configuration knobs, operator

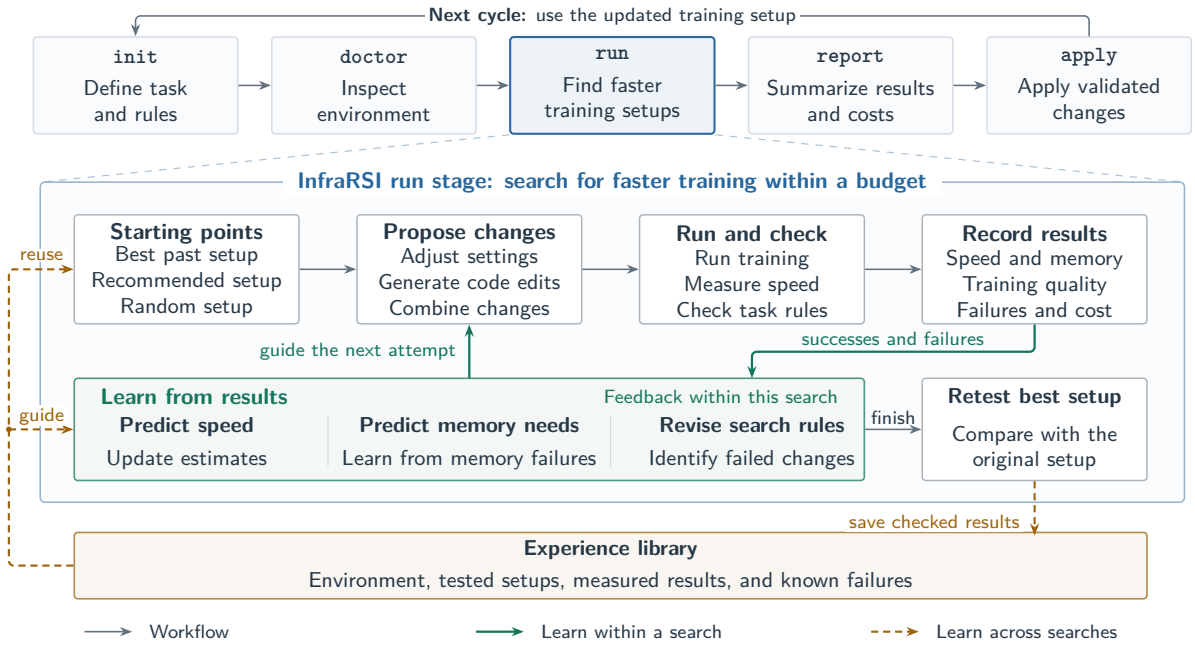

Figure 1 | **InfraRSI method overview.** The five-stage lifecycle defines task rules, inspects the environment and baseline performance, searches for changes, reports results, and applies validated changes. The expanded `run` stage shows InfraRSI searching over training settings and code edits generated by a language model. Measurements and failures update speed and memory predictions and guide the next attempt. The best setup is retested against the original before checked results are saved as reusable experience. Later searches use this experience to choose starting setups and initialize predictions. The updated training setup enters the next lifecycle.

and training-loop implementations, collective communication schedules, and device memory limits (Table 1). In this work, we study how to turn open-ended human commands and language-model proposals into verified training-system optimizations through a two-level self-improving loop—revising priors online within each tuning run while distilling verified runtime evidence into a cross-cluster experience bank under fixed task contracts. Jointly optimizing both launch configurations and training source code is essential because hardware-sensitive mechanisms such as FP8 quantization [23, 7], tensor and expert parallelism, graph compilation (`torch.compile`) [2], and FlashAttention [6, 35] interact non-linearly and often reverse their performance impact across GPU architectures and interconnect rooflines [42] (Table 3).

Yet applying unconstrained coding agents [41, 15, 16] or greedy local search loops directly to production training repositories exposes four recurring failure modes (Section 2). Open-ended code edits frequently introduce reward hacking, numerical drift, or timing shortcuts, achieving illusory speedups by altering effective batch sizes, deferring optimizer updates past the timing window, or applying unstable low-precision casts. At the same time, distributed clusters exhibit wide-spectrum measurement noise across two orders of magnitude, causing fixed acceptance thresholds either to promote random jitter on shared hosts or to discard valid communication overlaps and incremental code patches on quiet nodes. Guidance from zero-shot LLM priors is similarly unreliable: predicted speedups correlate poorly with actual hardware execution and often veto high-gain compilation or communication settings. Finally, single-incumbent search loops stall at out-of-memory (OOM) and compiler boundaries whenever a throughput-boosting setting exceeds device memory on its own and must be paired simultaneously with memory-saving knobs and compatible kernel patches.

To address these failure modes, we present **InfraRSI** (Figure 1, Section 3), the agentic harness that orchestrates a five-stage optimization workflow (`init` → `doctor` → `run` → `report` → `apply`). In InfraRSI, `init` locks editable repository scope and a fixed task contract over workload, loss, and timing invariants to prevent reward hacking, while `doctor` profiles baseline execution phases and calibrates paired step-time noise on the target cluster. The `run` stage executes **budgeted compositional search** (Section 4), a history-aware search controller organized around three mechanisms: (1) *Multi-Start Initialization and Global Time/Memory Models* (§4.1), which starts from diverse experience-seeded configurations and fits global execution-time and right-censored peak-memory models across all trials; (2) *Prior Calibration, Failure Attribution, and Bundle Completion* (§4.2), which revises miscalibrated LLM priors from live probes and pairs high-upside OOM configurations with model-priced memory savers; and (3) *High-Impact Knob Escape and Code Patch Stacking* (§4.3), which expands search around high-leverage launch settings while stacking verified code patches onto stable configurations. At the close of `run`, the harness verifies top candidates via independent back-to-back re-runs against the baseline (Table 2) and distills hardware-normalized runtime evidence into the cross-hardware experience bank (§5), after which `report` and `apply` audit source paths, account for failed-trial costs, and commit promoted changes.

We evaluate InfraRSI in open-code optimization campaigns across four dense and Mixture-of-Experts (MoE) LLM training workloads: Qwen3.5-9B on 8×H100, Gemma-4-26B-A4B MoE on 8×H200, Qwen3.8-27B on 8×H200, and two-node Qwen3.5-35B-A3B MoE on 16×H100 (Table 4, Figure 2). Across these workloads, InfraRSI achieves a **1.83**× geometric-mean speedup and lifts mean training Model FLOPs Utilization (MFU) from 17.4% to 30.0% (reaching up to 2.75× on Gemma-4-26B-A4B MoE), outperforming RE-Bench (1.37×, 25.1% MFU), Auto-Research (1.34×, 24.5% MFU), and knob-only OptiCo (1.23×, 21.3% MFU). Furthermore, cross-SKU experience transfer accelerates convergence and improves final throughput over cold-start search (Table 5), while offline receipt and tolerance audits confirm that our 16-invariant verifier reliably filters out invalid and reward-hacking executions across numerical regimes (Table 6). Our contributions are:

- **Unified Execution and Verification Contract (§3):** A five-stage multi-stack harness over launch configurations and source patches that couples 16 workload, numerical, and timing invariants (Table 2) with source-path auditing, full failed-trial cost accounting, and back-to-back baseline verification.
- **Budgeted Compositional Search (§4):** A stateful configuration and code search controller that uses Cholesky execution-time and censored Tobit peak-memory models to guide empirical prior calibration, failure-attributed bundle completion, and high-impact knob escape (Figure 1).
- **Cross-Hardware Experience Transfer and Audited Evaluation (§5, §6):** A structured experience protocol that transfers hardware-conditioned performance evidence across GPU architectures, validated across dense and MoE training workloads and numerical tolerance regimes (Tables 4–6, Figures 2–5).

2. Problem Definition and Empirical Motivation

2.1. Optimization Objective and Fixed Task Contract

Optimizing distributed training infrastructure differs fundamentally from algorithmic discovery [33, 25, 14] and isolated kernel generation [26, 16] (Table 1). In InfraRSI, each candidate combines launch configuration knobs and source-code patches to maximize training throughput under a strict *fixed task contract*: the model architecture, dataset, effective workload (tokens processed per optimizer update), allowed numerical precision menu, and optimizer update protocol are held

invariant. Each task contract declares the evaluation window, permitted numerical modes, and relative loss tolerance, while retaining the full GPU and API search cost of any failed or rejected candidate.

Evaluating a candidate under this contract separates two distinct verification questions: *short-horizon loss consistency* checks that the candidate's execution path remains numerically faithful within the declared precision menu and loss window, whereas *independent back-to-back verification* confirms that a candidate's speedup reproduces against a fresh reference execution beyond cluster measurement noise. Because distributed training mechanisms compete for finite device memory, streaming multiprocessors, and collective-communication overlap windows, individual optimizations combine sub-additively and frequently trigger steep feasibility boundaries, such as compilation and activation-checkpointing conflicts on Mixture-of-Experts models. Expressing both search cost and projected runtime savings in common GPU-hour units allows the harness to schedule and evaluate optimization trials directly.

Table 1 | Comparison of RSI paradigms across algorithmic synthesis, kernel generation, and training infrastructure.

Dimension	Algorithm / Code RSI [33, 25, 14]	Kernel Synthesis [26, 16]	Training Infra RSI (InfraRSI)
Granularity	Single heuristic or script	Isolated CUDA/Triton kernel [39]	Launch knobs & source-code patches (multi-stack)
Cost & Noise	ms–s; 10^3–10^4 runs; low noise	ms/op; 10^2–10^3 runs; low noise	Multi-GPU; 10–20 runs; 0.04%–6%+ noise
Equivalence	Test suite or val score	Tensor tolerance (L_∞/L_2)	Fixed task contract + source-path audit
Hacking Risk	Moderate (val overfitting)	High (caching, skipped sync)	Severe (prefetch skew, timer shift, precision drift)
HW Sensitivity	Hardware-agnostic logic	GPU-specific (SM, SRAM)	Roofline/topology-sensitive (H100/H200/H20)
Reusable Asset	Task-specific code pool	Op kernel library	Cross-hardware priors + audited patch sites

2.2. Evaluation Baselines and System Failure Modes

To evaluate open-code infrastructure search fairly, we compare InfraRSI against representative autonomous coding agents and configuration tuners, including Auto-Research [15], the RE-Bench modular agent scaffold [41], and knob-only OptiCo [3], under identical task contracts while retaining each baseline's native search rules. Across all campaigns, completed trial counts, GPU execution time, and token consumption are logged along the search trajectory, including all failed and audit-excluded attempts.

Audited runs across our four dense and MoE LLM workloads (Table 4) expose four recurring failure modes in unconstrained coding agents and greedy local tuners:

1. Reward hacking and timing-boundary skew. Without strict contract verification and source-path auditing, open-code agents frequently achieve illusory speedups via reward gaming [37] by altering effective work or timing boundaries. Common failure patterns include unauthorized low-precision casts that violate the declared precision menu, deferred auxiliary updates that push work past the timing window while masking runtime import errors behind a single receipt's validity flag, step-logging timer shifts that inflate native reported speed relative to wall-clock time, and aggressive low-precision or fused cross-entropy edits that degrade training loss (§3).

2. Wide-spectrum measurement noise. Across clusters and workloads, paired step-time noise spans more than two orders of magnitude, ranging from 0.04%–0.10% on dedicated 8×H100 and 8×H200 nodes to over 6% on shared hosts experiencing periodic data-loader or collective stalls. A hardcoded acceptance threshold (such as a fixed 2% gate) is too coarse on quiet clusters, where it discards valid 0.5%–1.5% communication overlaps and incremental source-code patches that

compound when stacked, yet still promotes random jitter on noisy hosts unless governed by calibrated cluster noise gates and independent back-to-back re-runs (§4.1).

3. Miscalibrated LLM quantitative priors. While frontier LLMs can inspect training repositories and propose qualitative mechanisms, their zero-shot speedup predictions correlate negatively with hardware execution (Spearman rank correlation $r \in [-0.49, -0.06]$). In addition, zero-shot priors issue *false-negative vetoes* on critical high-impact knobs: on Gemma-4-26B-A4B MoE (8×H200), the prior predicts a negligible +2.0% gain for graph compilation where empirical execution unlocks +55%; on two-node Qwen3.5-35B-A3B MoE (16×H100), it predicts a slight slowdown for increasing NCCL communication channels where execution yields a +10.2% gain. A search engine must therefore combine multi-start initialization with contrarian probing and empirical prior revision (§4.2).

4. Single-incumbent myopia at memory and compiler cliffs. Optimizing training infrastructure requires coordinating multiple launch knobs and code patches across steep memory and compatibility boundaries. Greedy single-incumbent search loops fail in three ways. First, they discard or repeat failures without attribution, repeatedly triggering identical out-of-memory crashes or blaming valid launch configurations when an exploratory source patch fails. Second, they abandon multi-knob synergies after single-knob OOMs: on Gemma-4-26B-A4B MoE, disabling activation recomputation alone exhausts device memory (+18 GiB shortfall) and enabling compilation with default fused kernels hangs, locking out tuners that cannot pair the memory-hungry optimization with a memory-saving partner (−14 GiB) and `FlexAttention` [8] to reach 2.75× (17.6% MFU). Third, they stall in local optima, as when Auto-Research plateaus at 1.29× on Gemma-4-26B-A4B MoE and spends its remaining trials circling that local basin (§4, §5).

3. System Architecture: The InfraRSI Harness

To address the failure modes in §2, InfraRSI decouples language-model hypothesis generation from measurement, verification, search, and experience storage. The harness pairs a declarative `infrarsi.yaml` task specification with lightweight stack adapters (< 15 hook lines) to expose a uniform execution and receipt interface across major distributed LLM training frameworks, including Megatron-Core [36, 24], FSDP2/TorchTitan [46, 18], DeepSpeed [30, 29], HF Trainer/TRL [43], and native PyTorch DDP.

3.1. Workflow and Multi-Stack Adapters

Every optimization campaign progresses through five deterministic CLI stages (Figure 1):

- **1. `init`:** Parses target repository ASTs, declares the task scope and `editable_files`, and locks SHA-256 digests of all `frozen_files` (dataset loaders, evaluation scripts, loss definitions, and harness timers).
- **2. `doctor`:** Executes reference baseline runs, trims warmup steps, calibrates cluster step-time noise, profiles execution phase breakdowns (forward, backward, optimizer, data loading, and collectives), probes interconnect bandwidth, and grades experience-bank prior coverage (§5).
- **3. `run`:** Executes InfraRSI's compositional search controller (§4) over candidate configurations and source patches, preserving raw execution receipts and full resource costs for every attempted candidate; at the end of search, it subjects the fastest valid candidates to fresh back-to-back verification re-runs against the reference baseline on the same GPU allocation, writes verified observations and experience packs back to the experience bank (§5), and saves `result.json`, `pack.yaml`, and `REPORT.md`.
- **4. `report`:** Renders and inspects the structured campaign report (`REPORT.md`), invariant verdicts, and resource accounting for candidate source-path auditing.

- **5. `apply`:** Reads the promoted candidate from `result.json`, checks target repository files for post-search working-tree conflicts, and writes the validated launch configuration and source diffs (optionally on a new git branch).

3.2. Measurement and Resource Accounting

InfraRSI represents each candidate as a joint specification of **launch configuration knobs** (CLI flags, environment variables, and configuration keys filtered by algebraic batch-conservation checks) and **source-code patches** that synthesize AST-validated unified diffs on `editable_files`, such as compiling block-sparse or chunked attention kernels [8, 2], fusing normalization, rotary-embedding [13], and AdamW [21] optimizer operators, and bucketing gradient-norm collectives. Every candidate evaluation and verification run enforces a fixed optimizer-update count, explicit warmup trimming, and a synchronized steady-state timing window.

To prevent survivorship bias, the harness tracks GPU hours across reference, search, promotion, and profiling stages alongside language-model token consumption, retaining the full GPU and API cost of every failed or audit-excluded candidate. Incomplete runs from transient host retries are flagged in the ledger, and migrating a workload to a new host requires establishing a fresh local baseline.

3.3. Semantic, Numerical, and Source-Path Verification

Within each fixed task contract, InfraRSI maximizes log speedup over the candidate space $\mathcal{A}$ subject to 16 deterministic invariants $\mathcal{I}_k$ across five verification layers:

$$\max_{a \in \mathcal{A}} \; y(a) = \ln \frac{T_0}{T(a)} \quad \text{s.t.} \quad \mathcal{I}_k(a) = \text{True}, \; \forall k \in \{1, \ldots, 16\}. \tag{1}$$

Task invariants verify execution completion and VRAM caps (L1), model parameters, sequence length, tokens per update, data digests, and `frozen_files` hashes (L2), actual optimizer step counts (L3), permitted precision menu and relative loss bounds under pre-declared `tolerance` regimes ranging from 0.1% to 50% (L4), and harness-to-script timing agreement (L5). For numerical verification (L4), all loss tolerances and adaptation rules are fixed in the task specification before search begins: by default, L4 bounds relative window-mean loss deviation within τ (adjusting to $3\sigma_\ell\sqrt{2/k}$ if reference repeats vary beyond τ, or enforcing per-update maximum deviation when exact batch order is verified; Table 8). Any candidate exceeding these pre-declared bounds is rejected while retaining its incurred search cost.

Alongside automated receipt checks, InfraRSI performs **candidate source-path auditing** before promoting a result. In our open-code evaluations (§6), source review excluded candidates that deferred auxiliary parameter updates to the subsequent step (which omitted the final update from the timed window and masked a runtime kernel import failure behind a single receipt's validity flag) or that introduced unauthorized low-precision casts outside the task's permitted precision menu, while confirming that compliant candidates execute all forward, backward, and optimizer steps synchronously inside the timed window.

3.4. Promotion and Delivered Speedups

Table 2 summarizes the five verification layers and their offline audit statistics (detailed predicates in Appendix A, Table 8; evaluation in §6.3). To ensure that search discoveries translate into reproducible system gains, promoted candidates undergo an independent back-to-back re-run against the reference configuration on the same GPU allocation.

Table 2 | **Five-layer, 16-item deterministic invariant verifier and offline re-verification ledger** (1,212 stored full-fidelity executions across 106 searches, complemented by source-path and contract audits). Triggers count every failed item of a rejected execution; primary attributes each rejected execution to its first failed layer (full predicates in Appendix A, Table 8).

Layer	Invariants	Core criterion	Offline triggers / primary
L1: Execution	`completes`, `no_errors`, `memory_cap`	Exit 0, full target update count completed, no NaN / OOM / NCCL / import error in main run, VRAM within cap	503 / 352 (`completes` 349, `no_errors` 153, `memory_cap` 1)
L2: Workload	`same_workload`, `same_data`, `same_model`, `frozen_files`	Same tokens per update and sequence length, per-update data digest, model digest and parameter count, frozen-file SHA-256 hashes; unverifiable required items reject	7 / 7 (unverifiable item 7)
L3: Schedule	`optimizer_steps`	Same actual optimizer-step count and synchronous auxiliary state updates inside the timed window	0 / 0 (offline; deferred updates caught in source audit)
L4: Numerical	`loss_tol`, `grad_norm_tol`, `precision`	Pre-declared task loss rule (window-mean within tolerance, or strict per-update max deviation when required); permitted precision menu enforced	42 / 41 (`loss_tol` 42; 39 pass without L4)
L5: Timing & Pair	`timing_consistent`, `speed_reproduces`	Harness wall-clock gain agrees with reported step-time gain ($\lvert\Delta \ln\rvert \leq 0.03$, flagging finite-prefetch or log-boundary skew); promotion requires back-to-back fresh re-test beyond calibrated noise	158 / 0 (`timing_consistent` 158; unpromoted runs deliver 1.00×)

Offline re-verification: 1,212 executions, 400 rejected (33.0%, 710 triggers), 812 valid; verifier unit suite reproduces 40/40 expected verdicts (28 perturbed rejected, 12 benign passed). Excluded candidates retain full API and GPU costs in the search ledger.

When no candidate clears all verification and promotion gates, the harness retains the reference configuration and reports a delivered speedup of 1.00×, preserving all unpromoted and excluded trials in the audit log.

4. Budgeted Compositional Search

Under practical multi-GPU search budgets, measurement jitter, miscalibrated language-model priors, steep memory and compiler-conflict cliffs, local optima, and late-stage context growth frequently stall naive proposal loops (§2). In each step of the `run` stage, InfraRSI reads and updates global execution-time and memory models across all completed trials (§4.1), revises language-model priors using live measurements and failure blame (§4.2), and proposes coordinated knob bundles and source patches (§4.3; Figure 1).

4.1. Multi-Start Initialization and Global Time/Memory Models

Online Noise Calibration and Phase-Headroom Ceiling. To guard against just-in-time compilation warmup [2, 39] and periodic data-loader or allocator jitter, the `doctor` stage trims initial warmup steps and measures each run's post-warmup steady-state window mean step time for candidate scoring, MFU accounting, and final promotion, while using robust median and split-half statistics inside the search loop so that transient single-step spikes do not distort surrogate updates. Across runs, InfraRSI estimates paired log-step-time noise σ_{pair} by pooling paired reference re-run differences with within-run split-half deviations (floored by the within-window white-noise floor), using $\max(0.02, 2\sigma_{\text{pair}})$ to govern big-knob selection and stall detection during search and requiring fresh back-to-back promotion re-runs to exceed the paired relative margin $\exp(2\sigma_{\text{pair}}) - 1$. This protocol measures paired baseline noise as low as 0.04%–0.10% on dedicated 8×H100 and 8×H200 nodes. Using the `doctor` stage's profiled operator-phase breakdown (forward, backward, optimizer, and collectives), InfraRSI also caps

single-lever log-speedup predictions via an Amdahl phase-headroom ceiling [1], bounding each lever's predicted gain by the profiled time share of the execution phases it targets.

Multi-Start Full-Configuration Initialization. Infrastructure search spaces partition into disconnected basins separated by out-of-memory boundaries and compiler incompatibilities. Starting solely from the unmodified default configuration wastes a large fraction of the trial budget flipping basic switches one by one, whereas trusting a single model-proposed bundle risks an immediate crash if any pair of levers conflicts. InfraRSI therefore dedicates its first three full trials to probing complementary complete configurations (Figure 1). The first start warm-starts from the best verified full configuration in the most similar hardware and model environment in the experience bank via canonical mechanism normalization. The second start probes the prior model's highest-rated full configuration that differs from the first start in at least one high-impact knob, promoting configuration diversity. The third start executes a uniform random legal configuration filtered by active conflict rules to anchor global speed and memory variance, with the first fitting run establishing the memory baseline. In our Qwen3.5-9B (8×H100) run, the experience-seeded first start reaches 1.55× on Trial 1 before subsequent search advances to 1.73×.

Global Time and Censored Memory Models. Because configuration knobs and source patches interact non-monotonically across speed and memory, stepping greedily from only the latest incumbent deadlocks whenever an early choice blocks a downstream optimization. InfraRSI instead maintains two global models fitted across the full trial history and checkpoints search state after each trial. A **Cholesky Bayesian linear time surrogate** models log-speedup over indicator features for non-reference knob settings, verified source-code patches, and enabling pairs, updating the posterior precision matrix in closed form while bounding prior variance. In parallel, a **right-censored Tobit memory ledger** [40] models per-GPU peak memory $\hat{m}(a)$ (in GiB) for candidate a on a device of capacity C_{card} (with observation noise $\sigma_{\text{obs}} = 0.02\,C_{\text{card}}$) across three execution regimes: (i) *exact Gaussian residuals* $\hat{m}(a) - m_{\text{peak}}$ on completed runs with measured peak memory m_{peak}; (ii) *right-censored Tobit likelihoods* enforcing $\hat{m}(a) \geq m_{\text{oom}}$ on out-of-memory trials, where the failed allocation shortfall establishes the lower bound m_{oom} on required memory; and (iii) *interval-censored likelihoods* bounding peak demand within $[0.90\,C_{\text{card}},\, C_{\text{card}}]$ when near-capacity allocator thrashing occurs. Fitting the posterior memory parameters via damped Newton steps with Laplace covariance yields calibrated memory feasibility probabilities and per-setting memory estimates (in GiB) that distinguish memory spenders from memory savers.

4.2. Prior Calibration, Failure Attribution, and Bundle Completion

Contrarian Big-Knob Probing and Empirical Prior Revision. Zero-shot language-model priors correlate poorly with actual multi-GPU execution (Spearman rank correlation $r \in [-0.49, -0.06]$) and frequently issue false-negative vetoes on high-leverage compiler or communication flags. To prevent miscalibrated priors from permanently hiding effective levers, InfraRSI applies empirical-Bayes shrinkage [10] across both action types: configuration-knob priors are regularized by their prior spread ($\sigma_0 \geq \max(\sigma_{\min}, 0.5|\mu_0|)$) and updated in closed form by the Cholesky time surrogate as observations arrive, while language-model code-patch gain predictions are scaled by a conservative shrinkage factor κ_0 ($\mu_{\text{patch}} = \kappa_0 \hat{\mu}_{\text{LLM}}$). Alongside Bayesian posterior updating, InfraRSI actively schedules **contrarian empirical probes** whenever a high-impact knob is rated low or vetoed by the prior yet exhibits high uncertainty whose two-sigma upside clears the big-knob threshold. InfraRSI tests such levers against the incumbent by reverting any conflicting setting to reference and automatically attaching the top estimated memory-saving setting when memory feasibility is low. After repeated configuration failures or any failure of

a high-confidence setting, InfraRSI also triggers a structured **empirical prior revision** carrying measured step times, peak memory, and out-of-memory shortfalls, directly updating the prior levels (μ_0, σ_0), refreshing the time surrogate's prior parameters and the memory ledger's declared GiB priors, and re-solving the posterior in place. As shown in §2, this probing and revision loop recovers high-gain compilation and communication settings (+55% and +10.2%) that zero-shot priors initially rate near zero or negative.

Failure Blame Attribution Across Memory, Conflicts, and Source Patches. InfraRSI logs out-of-memory events, numerical violations, and runtime crashes separately in the ledger. When a non-reference setting triggers an OOM, it is tagged as requiring a memory-saving partner covering at least the measured GiB shortfall, and is excluded only if it still fails across multiple distinct savers. For non-memory crashes, hangs, or numerical divergence, InfraRSI bans only the minimal conflict tuple (the smallest co-occurring pair or triple of settings absent from all passing runs) while keeping each individual setting eligible in other combinations. Failures during code exploration are attributed to the candidate patch rather than the underlying launch configuration.

Bundle Completion and Conflict Backtracking. Whenever a configuration fails due to insufficient memory despite high surrogate-predicted speedup, InfraRSI retains it in a high-upside failure buffer and performs bundle completion, pairing the configuration with reference-held memory savers that cover the measured GiB shortfall while reverting any conflicting knob to reference. For example, on Gemma-4-26B-A4B MoE, bundle completion pairs disabling activation recomputation (+18 GiB shortfall) with disabling Liger fused cross-entropy [13] (−14 GiB) and enabling `torch.compile` (§6.1). Similarly, when a high-prior lever conflicts with an earlier accepted baseline, conflict backtracking re-evaluates the lever directly on the clean root reference, migrates the incumbent once verified, and retains compatible settings on top of the updated baseline.

4.3. High-Impact Knob Escape and Code Patch Stacking

Focusing Exploration on the Big-Knob Set. When search needs to backtrack, probe contrarian directions, or jump out of a local basin, perturbing minor knobs (such as memory pinning or worker thread counts whose effect is below 1%) cannot shift the hardware roofline or free enough memory to unlock major speedups, and risks attributing gains to sub-noise fluctuations. After every observation, InfraRSI explicitly partitions the configuration space to isolate the **Big-Knob set** $\mathcal{K}_{\text{big}}$, defined as all configuration knobs whose prior ($\hat{y}_{\text{prior}}$), surrogate-posterior ($\hat{y}_{\text{surr}}$), or empirical ($\bar{y}_{\text{emp}}$) single-knob log-speedup magnitude on at least one setting clears $\max(0.02,\, 2\sigma_{\text{pair}})$ ($\approx$ 2%). All structural exploration operators (initial probe diversification, contrarian probing, trust-region escape, tabu deduplication, and warm restarts) operate over $\mathcal{K}_{\text{big}}$.

High-Impact Trust-Region Escape and Candidate Deduplication. Whenever two consecutive full trials fail to improve the incumbent beyond the noise gate, InfraRSI enters escape mode over $\mathcal{K}_{\text{big}}$. It dynamically expands the minimum trust-region distance from the incumbent from one to three big knobs to force multi-knob leaps out of the local basin, extrapolates further along ordered axes after improving steps, and performs periodic warm restarts from the best historical run of a distinct configuration class. To prevent wasted evaluations, InfraRSI also enforces big-knob tabu filtering and registers every proposed configuration-and-patch signature in the visited set *before* execution, cleanly separating pre-execution rejections from completed trials and preventing repetitive proposal loops.

Code Patch Stacking and Trajectory Summarization. When open-code editing is enabled, late-stage configuration proposals whose two-sigma upside falls below $\max(0.02, 2\sigma_{\text{pair}})$ yield

their evaluation slots to source-code synthesis, and verified patches with small individual gains are stacked and tested together in the spirit of group testing [9] once their combined log-gain clears the noise gate. To keep prompts bounded over long trajectories, InfraRSI retains full execution traces for the fastest and most recent runs while summarizing older trials into compact single-line records (~ 28k characters). In addition, when a candidate refines launch knobs on top of an already verified patch stack, unmodified inherited diff lines are not double-counted against the per-step line limit.

5. Empirical RSI Loop: Experience Protocol and Cross-Hardware Transfer

Reusing optimization history across training jobs requires converting raw execution traces into portable priors without inducing negative transfer across GPU generations. Because hardware roofline shifts alter effect magnitudes and signs while different frameworks expose distinct configuration flags, InfraRSI normalizes observations into a canonical experience schema weighted by hardware and workload similarity.

5.1. Cross-Hardware Rooflines and Sign Flips

Table 3 | **Cross-hardware scaling and sign flips in the initial seed corpus (11 seed packs, 191 environments, 698 observations, 543 effects)** across H20 (4.0 TB/s, 148 TF), H100 (3.35 TB/s, 989 TF), and H200 (4.8 TB/s, 989 TF).

Model	Mechanism	H20 (148 TF)	H100/200 (989 TF)	Roofline Cause & Rule
Qwen3-8B	`precision=fp8`	+58% (+54% TP2)	+30% H200 (+3% TP2)	Compute-bound H20 vs. comm-bound H200; κ scaling.
Qwen3-8B	`tp=2`	+24% (1.24×)	−4% (0.96× H100)	**Sign flip:** AllReduce overhead exceeds GEMM gain.
Qwen3-8B	`te_fp8_attn`	+6% (1.06×)	−8% (0.92× H200)	**Sign flip:** H200 BF16 FlashAttention-3 beats FP8 cast.
Dense LLM	`compile`	+20–29% (+5% TP2)	+5–10% (+40% H200)	Amortizes H20 launch; fuses H200 TP2 collectives.
Qwen3-30B MoE	`alltoall+grouped_gemm`	+92% (1.92×)	+85–110% (H200)	Consistent `kernel` gain on high-bandwidth NVLink.

Across our multi-cluster corpus spanning Hopper-generation H20, H100, and H200 GPUs on dense and MoE large language model workloads [44] (Table 3), end-to-end training speedups scale sub-linearly with peak device FLOPS because activation recomputation, MoE token routing, and memory-bound elementwise operations dilute raw GEMM throughput. Crossing the compute-to-bandwidth ridge point [42] also induces systematic *magnitude compression and sign flips* that break naive configuration replay. Compute-relief levers such as Transformer Engine FP8 execution [23] and tensor parallelism yield substantial gains on compute-bound H20 nodes, but their benefits compress sharply or turn negative on H100 and H200 once faster Tensor Cores shrink GEMM time enough that quantization cast overhead, optimized BF16 FlashAttention-3 kernels [35], or tensor-parallel AllReduce collectives dominate the training step. Compilation and communication-fusion effects similarly depend on interconnect topology and operator structure: graph compilation [2] amortizes kernel launch overhead on H20 and fuses tensor-parallel communication on H200, whereas NVLink-local MoE grouped GEMMs and all-to-all dispatch optimizations transfer consistently only within matching interconnect and model-architecture domains.

5.2. Structured Experience Protocol

To make empirical observations portable across heterogeneous repositories, InfraRSI structures experience records around three core entities: the **Environment**, capturing hardware rooflines, node and GPU topology, interconnect fabric, software versions, model dimensions, and parallel layout; the **Observation**, recording complete knob configurations, step time, peak device

memory, numerical verification conditions, and 16-invariant verdicts alongside explicit out-of-memory shortfalls and minimal conflict tuples; and the **Effect**, storing paired log-speedup, calibrated noise floor, conditioning dependencies, and physical mechanism class (`kernel` versus `balance`). A two-tier normalizer maps framework-specific flags (such as fused AdamW [21] or mixed-precision switches) across stacks into **71 canonical mechanisms**.

To transfer priors safely across clusters, InfraRSI filters and ranks each source record against the target environment using a compatibility scope check and an environmental distance $d \geq 0$. The scope check requires matching training frameworks for all records and grades an effect as `covered` when the model family and sequence-length bucket match (and, for topology-sensitive `balance` mechanisms, only when the GPU SKU, node count, per-node GPU count, multi-node interconnect fabric, parameter-scale bucket, and parallel layout also match), downgrading partial matches to `partial` and discarding incompatible records. Eligible records are then ordered lexicographically by coverage level (`covered` before `partial`), readability, evidence tier, and environmental distance d, which sums indicator penalties for GPU SKU and node-count mismatches, log-ratio differences $|\ln(a/b)|$ in parameter count and sequence length, and 0.25× reference-state mismatches. Environmental similarity is used strictly to *filter and rank prior evidence* (grading target levers into covered, null, partial, and unobserved states), whereas *actual target observations* govern posterior estimates during search via Bayesian surrogate updating and empirical prior revision [10] (§4.2). The corpus statistics in Table 3 (11 seed packs across 191 environments, 698 observations, 543 effects, and 71 canonical mechanisms) reflect the initial seed bank prior to our evaluation runs.

5.3. Experience Retrieval and Write-Back

Where Experience Enters Search. Within InfraRSI, retrieved experience packs enter the search loop at two explicit points: they directly drive the first start of the three-start initialization (§4.1) by mapping the highest-similarity verified winning configuration onto the target task's complete knob table, and they initialize the prior means and variances of the Cholesky Bayesian time surrogate as well as the per-setting GiB priors and conflict constraints of the right-censored Tobit memory ledger. In cross-hardware transfer evaluations (§6.2), retrieval excludes prior runs from the same workload and GPU SKU so that warm starts reflect cross-hardware priors rather than self-replay.

Verified Write-Back. At the end of a tuning run, observations that pass all verification invariants, numerical tolerances, and paired noise gates are packaged into a new experience pack and written back to the bank. In our controlled transfer evaluation on Qwen3.5-9B (8×H100) (§6.2, Table 5), cross-SKU experience improves final speedup from 1.70× to 1.82× and reaches the 90% peak log-speedup threshold at Trial 9 (2.97 GPU · h), whereas cold-start and unfiltered retrieval do not reach that threshold within the search budget.

6. Experimental Evaluation

We evaluate InfraRSI in three parts: (1) end-to-end training speedup and Model FLOPs Utilization (MFU) across four dense and MoE LLM workloads (§6.1, Table 4, Figures 2–3); (2) cross-hardware experience transfer (§6.2, Table 5, Figure 4); and (3) an independent offline verifier audit alongside numerical tolerance and noise-floor sensitivity studies (§6.3, Table 6, Figure 5).

6.1. End-to-End Speedup and MFU Across LLM and MoE Workloads

We evaluate InfraRSI (§4.3) against open-code Auto-Research [15], the METR RE-Bench agent scaffold [41], and knob-only OptiCo [3] across four dense and MoE LLM workloads [27, 11,

28]: Qwen3.5-9B (8×H100), Gemma-4-26B-A4B MoE (8×H200), Qwen3.8-27B (8×H200), and Qwen3.5-35B-A3B MoE (16×H100, 2 Nodes). Table 4 and Figures 2–3 report end-to-end training speedup and Model FLOPs Utilization (MFU) across all four workloads.

Table 4 | **End-to-end training speedup (×) and Model FLOPs Utilization (MFU, % of non-sparse BF16 peak at** 989.5 **TFLOP/s per GPU) across four dense and MoE LLM workloads.**

Workload & Hardware	Base MFU	OptiCo	Auto-Research	RE-Bench	InfraRSI	Key Discovered Knob + Code Combination
Qwen3.5-9B (8×H100)	25.3% (1.62 s)	1.24× (31.5%)	1.53× (38.6%)	1.62× (40.9%)	**1.73**× (**43.8**%)	Memory-ledger saver pairing (Liger frees VRAM for recompute-off + micro-batch 2) + fused normalization
Gemma-4-26B-A4B MoE (8×H200)	6.4% (3.38 s)	1.32× (8.4%)	1.29× (8.3%)	1.33× (8.5%)	**2.75**× (**17.6**%)	Synergistic memory completion (`torch.compile` + Liger-off + recompute-off) + cached `FlexAttention`
Qwen3.8-27B (8×H200)	29.3% (4.50 s)	1.19× (35.0%)	1.39× (40.8%)	1.38× (40.6%)	**1.55**× (**45.4**%)	Memory-saver pairing + fused RMSNorm/RoPE/SwiGLU kernels + selective activation recomputation
Qwen3.5-35B-A3B MoE (16×H100, 2 Nodes)	8.7% (1.89 s)	1.17× (10.3%)	1.17× (10.2%)	1.17× (10.3%)	**1.51**× (**13.2**%)	Contrarian 32-channel NCCL tuning + composite patch overlay (fused SwiGLU + bucketed grad norm)
4-Workload Mean (Geomean × / Mean MFU)	17.4%	1.23× (21.3%)	1.34× (24.5%)	1.37× (25.1%)	**1.83**× (**30.0**%)	Consistent gains across single- and multi-node dense and MoE training

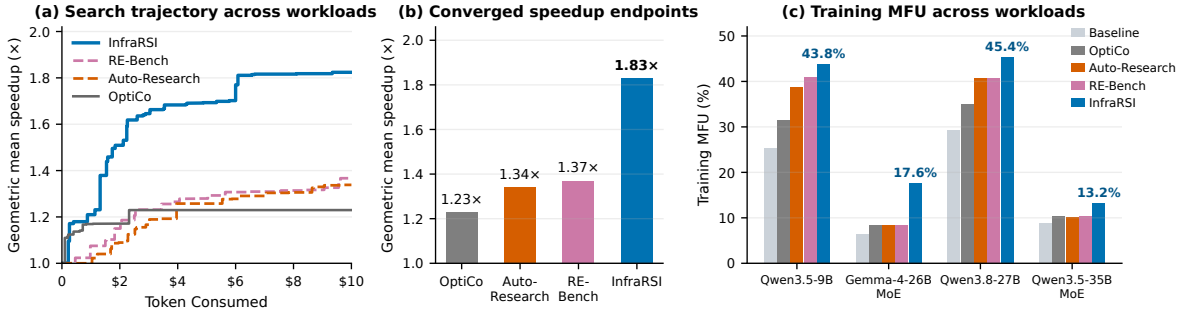

Figure 2 | **Search trajectories, final speedup endpoints, and training MFU across the four dense and MoE LLM workloads. (a)** Speedup trajectory versus token consumed across all compared methods. **(b)** Final geometric-mean speedup endpoints across the four workloads (1.83× for InfraRSI vs. 1.37× for RE-Bench, 1.34× for Auto-Research, and 1.23× for OptiCo). **(c)** Training MFU (% of dense BF16 peak) across workloads.

Overall End-to-End Speedup Performance. Across the four workloads, InfraRSI achieves a 1.83× geometric-mean speedup, consistently outperforming open-code RE-Bench (1.37×), Auto-Research (1.34×), and knob-only OptiCo (1.23×) on every task (Table 4, Figure 2). Knob-only tuning plateaus early because launch flags alone cannot remove operator-level bottlenecks, whereas single-incumbent coding agents stall when memory-hungry configurations trigger OOMs or compiler conflicts before compatible source patches can be stacked.

Model FLOPs Utilization (MFU) and Physical Throughput. We compute Training MFU as MFU $= F_{\text{update}}/(N_{\text{GPU}} \cdot P_{\text{BF16,dense}} \cdot t_{\text{update}})$ against the non-sparse Hopper BF16 peak (989.5 TFLOP/s per GPU), using post-warmup mean step time t_{update} (updates 11–30) and PaLM dense-equivalent active FLOPs F_{update} [5] (6× active matrix parameters per token plus attention FLOPs, excluding recomputation, communication, and unrouted experts). Overall, InfraRSI lifts mean MFU from 17.4% to 30.0%, pushing compute-bound dense models above 43% MFU and substantially improving utilization on routing- and communication-bound MoE workloads by jointly eliminating redundant activation recomputation, kernel launch overhead, and collective communication bottlenecks.

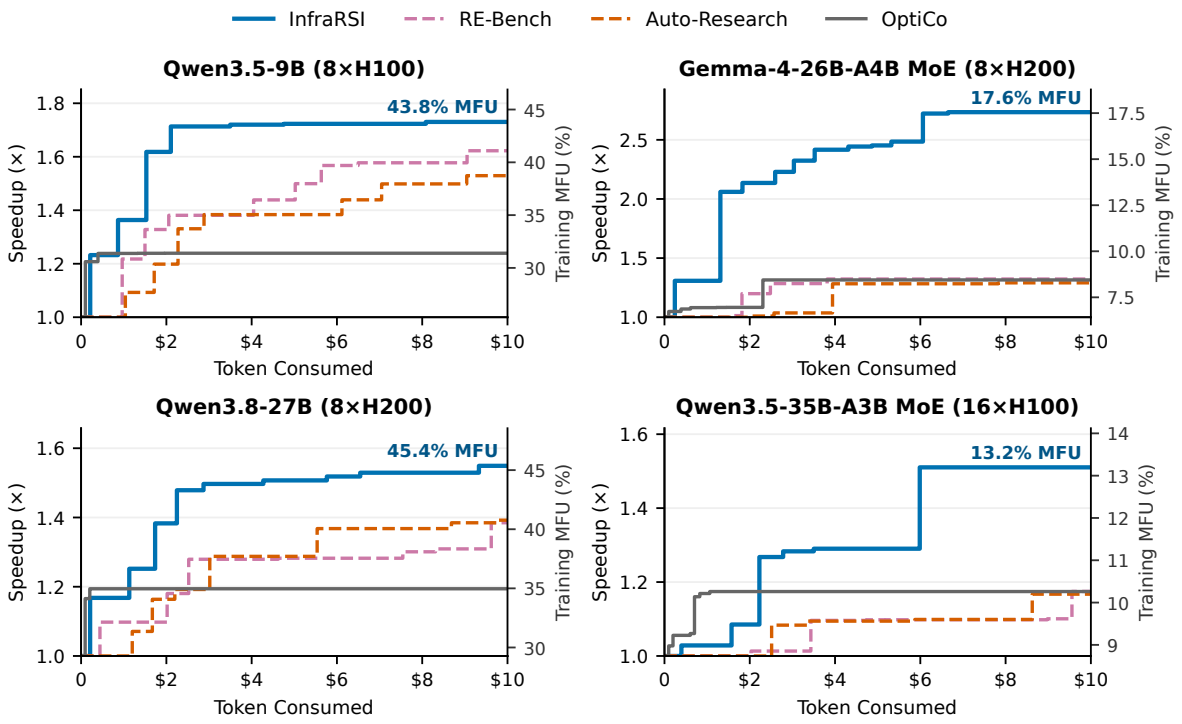

Figure 3 | **Per-workload search trajectories across the four dense and MoE LLM workloads,** plotting speedup (×) on the left axis and training MFU (%) on the right axis against token consumed.

Coordinated Knob and Source-Patch Discoveries. The per-workload trajectories in Figure 3 illustrate how compositional search coordinates configuration knobs with source-code patches under numerical and timing guardrails. On Gemma-4-26B-A4B MoE (8×H200), disabling activation recomputation in isolation triggers out-of-memory failures and enabling `torch.compile` conflicts with the default Liger kernel configuration [13]; InfraRSI's censored memory model identifies disabling Liger as a 14 GiB memory saver, bans the conflicting pair, and completes the three-way feasible knob combination while synthesizing a compiled `FlexAttention` [8] patch that caches block-sparse masks across the 25 sliding-window layers and a chunked causal attention kernel for the 5 global layers (2.75×, 17.6% MFU). On two-node Qwen3.5-35B-A3B MoE (16×H100), contrarian knob exploration unlocks 32-channel InfiniBand NCCL communication while composite patch overlays stack fused SwiGLU, fast zero-grad, and bucketed gradient-norm all-reduce kernels (1.51×, 13.2% MFU). On dense Qwen3.8-27B (8×H200) and Qwen3.5-9B (8×H100), pairing memory-saving configuration knobs with fused normalization, rotary embedding, and MLP kernels lifts final speedups to 1.55× (45.4% MFU) and 1.73× (43.8% MFU), respectively.

6.2. Cross-Hardware Experience Transfer

Table 5 and Figure 4 evaluate cross-hardware experience transfer on Qwen3.5-9B (8×H100). We compare cold start (no prior experience packs) and all eligible packs against cross-SKU transfer, which restricts both structured experience packs and the transfer brief to evidence harvested on other GPU architectures while excluding same-task, same-SKU history. When experience transfer is active, the initial trial warm-starts a knob configuration from the transferred winner and seeds priors into the time surrogate and memory ledger. We measure both the final speedup and the number of trials and GPU-hours required to reach 90% of the peak log-speedup ($\ln(S) \geq 0.9 \ln(S^\star)$).

Table 5 | **InfraRSI cross-hardware experience transfer on Qwen3.5-9B (8×H100).** Cross-SKU transfer filters both experience packs and the transfer brief to evidence harvested on other hardware SKUs while excluding same-task, same-SKU runs; the initial trial warm-starts a knob configuration and seeds priors for the time surrogate and memory ledger. Convergence speed is measured by the trials and GPU-hours required to reach 90% of the peak log-speedup ($\ln(S) \geq 0.9 \ln(S^\star)$).

Target Workload & Hardware	Experience Condition	First Trial	Final Speedup	Trials to 90% Peak	GPU-h to 90% Peak
Qwen3.5-9B (8×H100)	Cold start (no prior packs)	1.22× (Trial 2)	1.70×	> 15 (censored)	—
Qwen3.5-9B (8×H100)	All eligible packs	1.24× (Trial 1)	1.70×	> 17 (censored)	—
Qwen3.5-9B (8×H100)	Cross-SKU transfer	1.23× (Trial 1)	**1.82×**	**9**	**2.97**

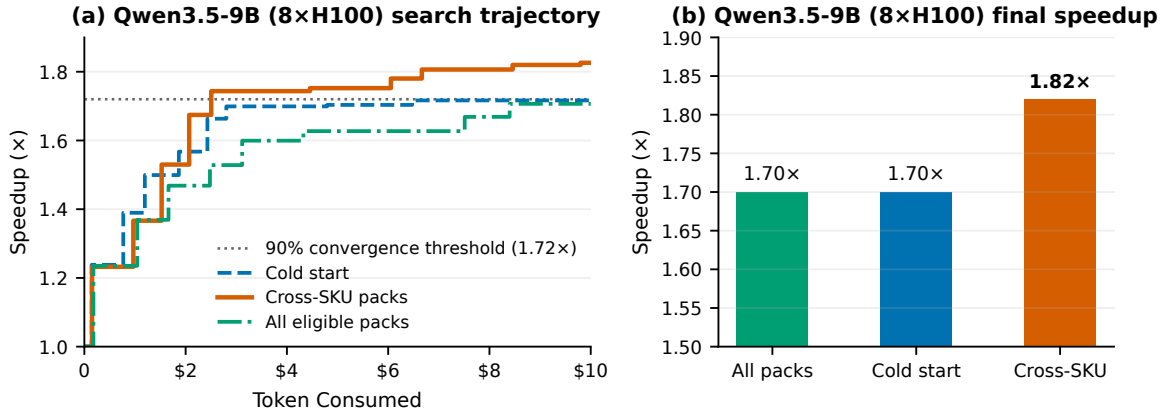

Figure 4 | **InfraRSI cross-hardware experience transfer trajectories on Qwen3.5-9B (8×H100).** Curves plot speedup against token consumed; the dashed horizontal line marks the 90% peak log-speedup threshold ($\ln(S) \geq 0.9 \ln(S^\star)$).

As shown in Table 5, cross-SKU experience transfer improves both final speedup and search convergence on Qwen3.5-9B (8×H100): it lifts the final speedup from 1.70× (under cold start and all eligible packs) to 1.82× and is the only condition to cross the workload-wide 90% peak threshold within the search budget (Trial 9, 2.97 GPU · h), whereas both cold start (> 15 trials) and all eligible packs (> 17 trials) are right-censored.

6.3. Offline Verifier Audit and Tolerance Sensitivity

Finally, Table 6 and Figure 5 evaluate our independent offline verifier audit (Part A) alongside our numerical tolerance and fixed-noise ablation on Qwen3.5-9B (8×H100) (Part B).

As shown in Table 6 (Part A), the released offline verifier test suite achieves 40/40 (100%) agreement with ground-truth expectations (rejecting all 28 single-perturbation receipts and accepting all 12 benign variants), while rejecting unsigned, post-signing edited, or schema-mismatched HMAC-SHA256 receipts. Re-judging all 1,212 stored executions with the 16-item verifier rejects 400 executions (33.0%), including 39 executions (3.2%) that pass all non-numerical checks and require the numerical loss and gradient verification layer (L4) to catch. In the live sensitivity study on Qwen3.5-9B (8×H100) (Table 6, Part B; Figure 5), varying the window-mean loss tolerance across 0.10%, 3.00% (default), and 50.00% or substituting a fixed 2.00% noise floor produces stable convergence: all four arms reach the common 90% peak threshold ($S^\star = 1.75\times$) within 10–11 trials and achieve final speedups between 1.70× and 1.75× (6.02–7.05 GPU · h).

7. Related Work

Table 7 organizes prior work along the four dimensions that separate an unconstrained coding loop from a verified infrastructure optimization harness: (1) Proposal Models, (2) Search Con-

Table 6 | **Offline verifier audit (Part A) and numerical tolerance and fixed-noise ablation on Qwen3.5-9B (8×H100) (Part B). Part A** summarizes the independent offline receipt matrix, cryptographic receipt validation, and offline re-judging of 1,212 stored executions. **Part B** compares window-mean loss relative tolerance $\in \{0.10\%, 3.00\%, 50.00\%\}$ and a fixed 2.00% noise floor (at 3.00% tolerance) on Qwen3.5-9B (8×H100) using InfraRSI (common 90% peak reference $S^\star = 1.75\times$).

Part A: Independent Offline Verifier & Receipt Audit

Suite	Audit Scope & Test Cases	Accepted	Rejected	Agreement	Escapes w/o L4	Verified Invariants & Failure Modes Caught
Receipt Matrix	40 released system tests (28 single-perturbation + 12 benign)	12 benign	28 perturbed	40/40 (100%)	—	Halved/missing tokens, sequence length, step count, LR schedule, data order, early exit 0, NaN/frozen loss, OOM/traceback, memory cap, edited frozen files, clock skew, model digest
HMAC-SHA256	Cryptographic receipt integrity test	0	All invalid	100%	—	Rejects unsigned receipts, post-signing step-time edits, and mismatched receipt schemas
Historical Corpus	Offline re-judge of 1,212 stored executions	812 (67.0%)	400 (33.0%)	—	39 (3.2%)	39 executions pass non-numerical checks and require the L4 numerical loss/gradient layer to reject

Part B: InfraRSI Tolerance & Fixed-Noise Ablation on Qwen3.5-9B (8×H100, $S^\star = 1.75\times$)

Arm	Setting (Qwen3.5-9B, 8×H100)	Valid / Comp.	Final Speedup	90% Peak	Search Cost	Verification & Resource Accounting Notes
Strict Tol.	Relative tolerance 0.10%	13/16	1.70×	Trial 11	6.48 GPU · h	All candidates pass strict numerical and timing boundary audits
Default Tol.	Relative tolerance 3.00% (default)	14/17	1.74×	Trial 10	7.05 GPU · h	Default task contract; preserves full failed/excluded search costs
Relaxed Tol.	Relative tolerance 50.00%	13/17	**1.75×**	Trial 10	6.90 GPU · h	Admits wider loss variance within declared precision menu
Fixed Noise	Fixed 2.00% noise floor, tol. 3.00%	11/15	1.72×	Trial 11	6.02 GPU · h	Replaces calibrated cluster noise floor with fixed 2.00% gate

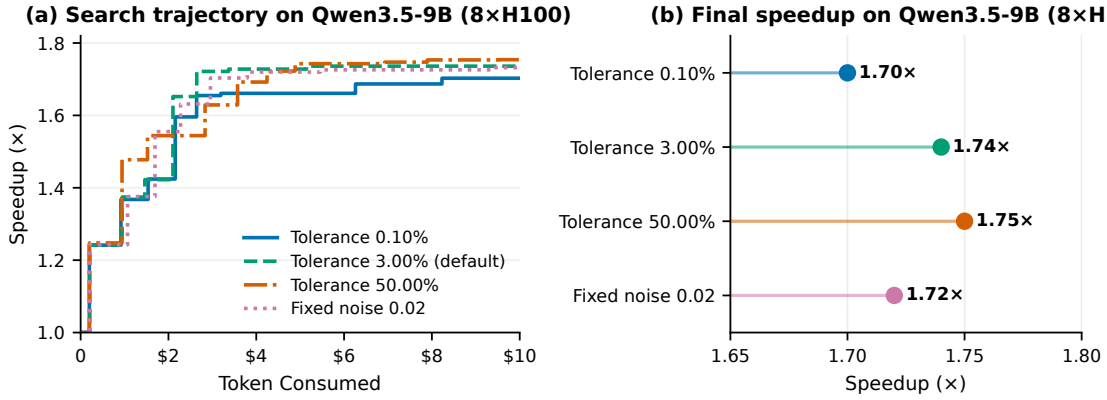

Figure 5 | **InfraRSI numerical tolerance and fixed-noise settings on Qwen3.5-9B (8×H100). (a)** Search trajectories across relative loss tolerances (0.10%, 3.00%, and 50.00%) and a fixed 2.00% noise floor (at 3.00% tolerance), with the common 90% peak threshold ($S^\star = 1.75\times$) marked. **(b)** Final speedups across the four ablation settings.

trollers, (3) Execution and Verification Interfaces, and (4) Experience Memory. Because prior studies evaluate disparate hardware generations, model sizes, and timing rules, Table 7 compares structural capabilities while Table 4 benchmarks representative methods under identical task contracts.

(1) Proposal Models. While the classical Gödel Machine [34] formulated self-referential modification via formal proof search, recent code-evolution and agent frameworks (including FunSearch [33], AlphaEvolve [25], Darwin Gödel Machine [45], AIDE [14], METR RE-Bench [41], and

Table 7 | **Positioning InfraRSI against existing paradigms across four architectural dimensions and eight operational properties** (✓: supported; △: partial; ×: absent; ○: underlying kernel/compiler primitive orchestrated by InfraRSI).

System / Paradigm	Primary Role	Stacks	Edit Scope	Contract	Noise Gate	Mem. Model	Escape	Code Audit	X-HW Prior
FunSearch / AlphaEvolve [33, 25]	Program evolution	×	Code	×	×	×	△	×	×
Darwin Gödel / AIDE [45, 14]	Agentic tree search	×	Code	×	×	×	△	×	×
TVM / Triton / Inductor [4, 39, 2]	Compiler / kernel IR	✓	○	✓	○	○	○	○	△
KernelBench / AI CUDA [26, 16]	Isolated kernel synth.	×	Kernel	×	×	×	×	△	×
Alpa / Galvatron / nnScaler [47, 22, 20]	Auto-parallelism planner	×	Graph/Plan	△	×	△	×	×	×
OptiCo [3]	Collaborative knob tuner	×	Knobs	×	×	×	×	×	×
Ray Tune / Black-box BO [19, 38]	Hyperparameter BO	✓	Knobs	×	△	×	×	×	×
METR RE-Bench [41]	Modular R&D agent scaffold	△	Free	△	×	×	×	×	×
Karpathy `autoresearch` [15]	Fixed-horizon LLM loop	×	Free	×	×	×	×	×	×
InfraRSI (Ours)	**Verified infra harness**	✓	**Knobs+Code**	✓	✓	✓	✓	✓	✓

`autoresearch` [15]) rely on the zero-shot coding priors of frontier LLMs to propose edits. In distributed LLM training, however, zero-shot speed predictions correlate poorly with measured hardware throughput and frequently veto high-gain optimizations (§2). InfraRSI decouples candidate generation from zero-shot model priors through empirical Bayes prior calibration [10], contrarian probing, and failure-driven prior revision.

(2) Search Controllers. Existing controllers split into static parallelism planners, fixed configuration-knob tuners, and single-incumbent free-form code-editing loops. Across distributed training stacks such as Megatron-LM [36, 24], DeepSpeed [30, 29], PyTorch FSDP [46], and TorchTitan [18], automatic parallelism planners like Alpa [47], Galvatron [22], and nnScaler [20] synthesize distributed execution plans over static computation graphs, whereas Bayesian optimization [38], tuning platforms such as Ray Tune [19], and multi-agent tuners like OptiCo [3] search predefined launch flags; none of them synthesizes source-level operator patches. Conversely, free-form coding agents such as `autoresearch` [15], RE-Bench agent scaffolds [41], and ResearchBoost-Agent [32] edit source files directly, but step greedily from the latest incumbent without a global memory model or combinatorial failure recovery, often stalling when individual edits trigger out-of-memory errors or compiler conflicts. InfraRSI coordinates launch knobs and source patches through multi-point initialization, a global step-time model paired with a censored memory model [40], bundle completion, plateau escape, and bounded context compression (§4; evaluated under identical contracts in §6.1, Table 4).

(3) Execution and Verification Interfaces. Kernel compilers, hand-optimized kernel libraries, and kernel-synthesis benchmarks, including TVM [4], Triton [39], PyTorch 2 Inductor [2], FlashAttention-2/3 [6, 35], FlexAttention [8], Liger Kernel [13], FP8 primitives [23], KernelBench [26], and AI CUDA Engineer [16], optimize isolated operators against unit-test shapes, whereas open-code training agents typically judge full scripts by reading self-reported stdout timers that are vulnerable to reward gaming [37, 17]. InfraRSI treats compilers and custom kernels as composable primitives within its staged verification harness (§3.4), combining the 16-item receipt verifier, source-code audits, and noise-calibrated promotion gates.

(4) Experience Memory. While online agent-serving platforms such as Reef [31] continually update deployed agents from inference feedback, most training-time hyperparameter tuners and coding agents either start from scratch on each cluster, rely on static hand-written skill cards [32], or append raw execution transcripts until context windows saturate. InfraRSI records reusable infrastructure findings as structured experience packs (§5) that normalize hardware rooflines, parallel topologies, verified log-speedups, numerical tolerances, and failure constraints across canonical optimization mechanisms. Harvested packs from external hardware SKUs warm-start

initialization and seed time and memory priors while excluding same-task, same-SKU runs (§6.2, Table 5).

8. Conclusion

In this paper, we propose the **InfraRSI Harness**, a harness that turns language-model proposals into verified distributed training optimizations by combining five-stage contract verification, history-aware compositional search over launch knobs and source patches, and cross-hardware experience transfer. Across single- and multi-node dense and Mixture-of-Experts training workloads, InfraRSI consistently improves end-to-end throughput and hardware utilization over existing configuration tuners and coding agents while preventing reward hacking.

Contributors

Yuchuan Tian[1,2], Haocheng Mei[1,2], Ye Yuan[1], Chao Xu[2], Boxun Li[3], Guohao Dai[4,3], Kai Han[1,†], Allen Chen[5,†], Yunhe Wang[1,†], Yu Wang[5,†]

[1]TokenRhythm Technologies [2]Peking University [3]Infinigence AI [4]Shanghai Jiao Tong University [5]Tsinghua University

[†]Corresponding authors.

References

[1] Gene M. Amdahl. 1967. Validity of the single processor approach to achieving large scale computing capabilities. In *Proceedings of the AFIPS Spring Joint Computer Conference*, pages 483–485.

[2] Jason Ansel, Edward Yang, Horace He, Natalia Gimelshein, Animesh Jain, Michael Voznesensky, Bin Bao, Peter Bell, David Berard, Evgeni Burovski, Geeta Chauhan, Anjali Chourdia, Will Constable, Alban Desmaison, Zachary DeVito, Elias Ellison, Will Feng, Jiong Gong, Michael Gschwind, Brian Hirsh, Sherlock Huang, Kshiteej Kalambarkar, Laurent Kirsch, Michael Lazos, Mario Lezcano, Yanbo Liang, Jason Liang, Yinghai Lu, C. K. Luk, Bert Maher, Yunjie Pan, Christian Puhrsch, Matthias Reso, Mark Saroufim, Marcos Yukio Siraichi, Helen Suk, Michael Suo, Phil Tillet, Eikan Wang, Xiaodong Wang, William Wen, Shunting Zhang, Xu Zhao, Keren Zhou, Richard Zou, Ajit Mathews, Gregory Chanan, Peng Wu, and Soumith Chintala. 2024. PyTorch 2: Faster machine learning through dynamic Python bytecode transformation and graph compilation. In *Proceedings of the 29th ACM International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS)*, volume 2, pages 929–947.

[3] Sheng Chen, Zhe Tang, Weixing Zhang, Fei Yang, Yuanyuan Wang, Tianlin Li, and Yang Liu. 2026. OptiCo: Adaptive distributed training optimization via collaborative agent reasoning. In *Proceedings of the 64th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 27847–27867.

[4] Tianqi Chen, Thierry Moreau, Ziheng Jiang, Lianmin Zheng, Eddie Yan, Haichen Shen, Meghan Cowan, Leyuan Wang, Yuwei Hu, Luis Ceze, Carlos Guestrin, and Arvind Krishnamurthy. 2018. TVM: An automated end-to-end optimizing compiler for deep learning. In *Proceedings of the 13th USENIX Symposium on Operating Systems Design and Implementation (OSDI)*, pages 578–594.

[5] Aakanksha Chowdhery, Sharan Narang, Jacob Devlin, Maarten Bosma, Gaurav Mishra, Adam Roberts, Paul Barham, Hyung Won Chung, Charles Sutton, Sebastian Gehrmann, Parker Schuh, Kensen Shi, Sasha Tsvyashchenko, Joshua Maynez, Abhishek Rao, Parker Barnes, Yi Tay, Noam Shazeer, Vinodkumar Prabhakaran, Emily Reif, Nan Du, Ben Hutchinson, Reiner Pope, James Bradbury, Jacob Austin, Michael Isard, Guy Gur-Ari, Pengcheng Yin, Toju Duke, Anselm Levskaya, Sanjay Ghemawat, Sunipa Dev, Henryk Michalewski, Xavier Garcia, Vedant Misra, Kevin Robinson, Liam Fedus, Denny Zhou, Daphne Ippolito, David Luan, Hyeontaek Lim, Barret Zoph, Alexander Spiridonov, Ryan Sepassi, David Dohan, Shivani Agrawal, Mark Omernick, Andrew M. Dai, Thanumalayan Sankaranarayana Pillai, Marie Pellat, Aitor Lewkowycz, Erica Moreira, Rewon Child, Oleksandr Polozov, Katherine Lee, Zongwei Zhou, Xuezhi Wang, Brennan Saeta, Mark Diaz, Orhan Firat, Michele Catasta, Jason Wei, Kathy Meier-Hellstern, Douglas Eck, Jeff Dean, Slav Petrov, and Noah Fiedel. 2023. PaLM: Scaling language modeling with Pathways. *Journal of Machine Learning Research*, 24(240):1–113.

[6] Tri Dao. 2024. FlashAttention-2: Faster attention with better parallelism and work partitioning. In *The Twelfth International Conference on Learning Representations (ICLR)*.

[7] DeepSeek-AI. 2024. DeepSeek-V3 technical report. *arXiv preprint arXiv:2412.19437*.

[8] Juechu Dong, Boyuan Feng, Driss Guessous, Yanbo Liang, and Horace He. 2025. FlexAttention: A programming model for generating optimized attention kernels. In *Proceedings of Machine Learning and Systems (MLSys)*.

[9] Robert Dorfman. 1943. The detection of defective members of large populations. *The Annals of Mathematical Statistics*, 14(4):436–440.

[10] Bradley Efron and Carl Morris. 1973. Stein's estimation rule and its competitors—an empirical Bayes approach. *Journal of the American Statistical Association*, 68(341):117–130.

[11] Gemma Team. 2026. Gemma 4 technical report. *arXiv preprint arXiv:2607.02770*.

[12] Jordan Hoffmann, Sebastian Borgeaud, Arthur Mensch, Elena Buchatskaya, Trevor Cai, Eliza Rutherford, Diego de Las Casas, Lisa Anne Hendricks, Johannes Welbl, Aidan Clark, Tom Hennigan, Eric Noland, Katie Millican, George van den Driessche, Bogdan Damoc, Aurelia Guy, Simon Osindero, Karen Simonyan, Erich Elsen, Jack W. Rae, Oriol Vinyals, and Laurent Sifre. 2022. Training compute-optimal large language models. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 35, pages 30016–30030.

[13] Pin-Lun Hsu, Yun Dai, Vignesh Kothapalli, Qingquan Song, Shao Tang, Siyu Zhu, Steven Shimizu, Shivam Sahni, Haowen Ning, and Yanning Chen. 2024. Liger Kernel: Efficient Triton kernels for LLM training. *arXiv preprint arXiv:2410.10989*.

[14] Zhengyao Jiang, Dominik Schmidt, Dhruv Srikanth, Dixing Xu, Ian Kaplan, Deniss Jacenko, and Yuxiang Wu. 2025. AIDE: AI-driven exploration in the space of code. *arXiv preprint arXiv:2502.13138*.

[15] Andrej Karpathy. 2026. `autoresearch`: AI agents running research on single-GPU nanochat training automatically. GitHub repository, `https://github.com/karpathy/autoresearch`.

[16] Robert Tjarko Lange, Aaditya Prasad, Qi Sun, Maxence Faldor, Yujin Tang, and David Ha. 2025. The AI CUDA Engineer: Agentic CUDA kernel discovery, optimization and composition. Technical report, Sakana AI.

[17] Robert Tjarko Lange, Qi Sun, Aaditya Prasad, Maxence Faldor, Yujin Tang, and David Ha. 2025. Towards robust agentic CUDA kernel benchmarking, verification, and optimization. *arXiv preprint arXiv:2509.14279*.

[18] Wanchao Liang, Tianyu Liu, Less Wright, Will Constable, Andrew Gu, Chien-Chin Huang, Iris Zhang, Wei Feng, Howard Huang, Junjie Wang, Sanket Purandare, Gokul Nadathur, and Stratos Idreos. 2025. TorchTitan: One-stop PyTorch native solution for production ready LLM pre-training. In *The Thirteenth International Conference on Learning Representations (ICLR)*.

[19] Richard Liaw, Eric Liang, Robert Nishihara, Philipp Moritz, Joseph E. Gonzalez, and Ion Stoica. 2018. Tune: A research platform for distributed model selection and training. *arXiv preprint arXiv:1807.05118*.

[20] Zhiqi Lin, Youshan Miao, Quanlu Zhang, Fan Yang, Yi Zhu, Cheng Li, Saeed Maleki, Xu Cao, Ning Shang, Yilei Yang, Weijiang Xu, Mao Yang, Lintao Zhang, and Lidong Zhou. 2024. nnScaler: Constraint-guided parallelization plan generation for deep learning training. In *Proceedings of the 18th USENIX Symposium on Operating Systems Design and Implementation (OSDI)*, pages 347–363.

[21] Ilya Loshchilov and Frank Hutter. 2019. Decoupled weight decay regularization. In *The Seventh International Conference on Learning Representations (ICLR)*.

[22] Xupeng Miao, Yujie Wang, Youhe Jiang, Chunan Shi, Xiaonan Nie, Hailin Zhang, and Bin Cui. 2023. Galvatron: Efficient transformer training over multiple GPUs using automatic parallelism. *Proceedings of the VLDB Endowment*, 16(3):470–479.

[23] Paulius Micikevicius, Dusan Stosic, Neil Burgess, Marius Cornea, Pradeep Dubey, Richard Grisenthwaite, Sangwon Ha, Alexander Heinecke, Patrick Judd, John Kamalu, Naveen Mellempudi, Stuart Oberman, Mohammad Shoeybi, Michael Siu, and Hao Wu. 2022. FP8 formats for deep learning. *arXiv preprint arXiv:2209.05433*.

[24] Deepak Narayanan, Mohammad Shoeybi, Jared Casper, Patrick LeGresley, Mostofa Patwary, Vijay Korthikanti, Dmitri Vainbrand, Prethvi Kashinkunti, Julie Bernauer, Bryan Catanzaro, Amar Phanishayee, and Matei Zaharia. 2021. Efficient large-scale language model training on GPU clusters using Megatron-LM. In *Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis (SC)*, Article 58.

[25] Alexander Novikov, Ngân Vũ, Marvin Eisenberger, Emilien Dupont, Po-Sen Huang, Adam Zsolt Wagner, Sergey Shirobokov, Borislav Kozlovskii, Francisco J. R. Ruiz, Abbas Mehrabian, M. Pawan Kumar, Abigail See, Swarat Chaudhuri, George Holland, Alex Davies, Sebastian Nowozin, Pushmeet Kohli, and Matej Balog. 2025. AlphaEvolve: A coding agent for scientific and algorithmic discovery. *arXiv preprint arXiv:2506.13131*.

[26] Anne Ouyang, Simon Guo, Simran Arora, Alex L. Zhang, William Hu, Christopher Ré, and Azalia Mirhoseini. 2025. KernelBench: Can LLMs write efficient GPU kernels? In *Proceedings of the 42nd International Conference on Machine Learning (ICML)*, PMLR, volume 267, pages 47356–47415.

[27] Qwen Team. 2026. Qwen3.5: Towards native multimodal agents. Model release, `https://huggingface.co/Qwen/Qwen3.5-9B`.

[28] Qwen Team. 2026. Qwen3.8-27B. Model release, `https://huggingface.co/Qwen/Qwen3.8-27B`.

[29] Samyam Rajbhandari, Jeff Rasley, Olatunji Ruwase, and Yuxiong He. 2020. ZeRO: Memory optimizations toward training trillion parameter models. In *Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis (SC)*, pages 1–16.

[30] Jeff Rasley, Samyam Rajbhandari, Olatunji Ruwase, and Yuxiong He. 2020. DeepSpeed: System optimizations enable training deep learning models with over 100 billion parameters. In *Proceedings of the 26th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining (KDD)*, pages 3505–3506.

[31] Reef Contributors. 2026. Reef: Infrastructure for continually self-improving agents. GitHub repository, `https://github.com/Human-Agent-Society/reef`.

[32] ResearchBoost Team. 2026. ResearchBoost-Agent: An autonomous agent that profiles, optimizes, and validates AI training code. GitHub repository, `https://github.com/AI4AI-for-Anything/ResearchBoost-Agent`.

[33] Bernardino Romera-Paredes, Mohammadamin Barekatain, Alexander Novikov, Matej Balog, M. Pawan Kumar, Emilien Dupont, Francisco J. R. Ruiz, Jordan S. Ellenberg, Pengming Wang, Omar Fawzi, Pushmeet Kohli, and Alhussein Fawzi. 2024. Mathematical discoveries from program search with large language models. *Nature*, 625:468–475.

[34] Jürgen Schmidhuber. 2007. Gödel machines: Fully self-referential optimal universal self-improvers. In Ben Goertzel and Cassio Pennachin, editors, *Artificial General Intelligence*, pages 199–226. Springer, Berlin, Heidelberg.

[35] Jay Shah, Ganesh Bikshandi, Ying Zhang, Vijay Thakkar, Pradeep Ramani, and Tri Dao. 2024. FlashAttention-3: Fast and accurate attention with asynchrony and low-precision. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 37.

[36] Mohammad Shoeybi, Mostofa Patwary, Raul Puri, Patrick LeGresley, Jared Casper, and Bryan Catanzaro. 2019. Megatron-LM: Training multi-billion parameter language models using model parallelism. *arXiv preprint arXiv:1909.08053*.

[37] Joar Skalse, Nikolaus H. R. Howe, Dmitrii Krasheninnikov, and David Krueger. 2022. Defining and characterizing reward gaming. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 35, pages 9460–9471.

[38] Jasper Snoek, Hugo Larochelle, and Ryan P. Adams. 2012. Practical Bayesian optimization of machine learning algorithms. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 25, pages 2951–2959.

[39] Philippe Tillet, H. T. Kung, and David Cox. 2019. Triton: An intermediate language and compiler for tiled neural network computations. In *Proceedings of the 3rd ACM SIGPLAN International Workshop on Machine Learning and Programming Languages (MAPL)*, pages 10–19.

[40] James Tobin. 1958. Estimation of relationships for limited dependent variables. *Econometrica*, 26(1):24–36.

[41] Hjalmar Wijk, Tao Lin, Joel Becker, Sami Jawhar, Neev Parikh, Thomas Broadley, Lawrence Chan, Michael Chen, Josh Clymer, Jai Dhyani, Elena Ericheva, Katharyn Garcia, Brian Goodrich, Nikola Jurkovic, Megan Kinniment, Aron Lajko, Seraphina Nix, Lucas Sato, William Saunders, Maksym Taran, Ben West, and Elizabeth Barnes. 2025. RE-Bench: Evaluating frontier AI R&D capabilities of language model agents against human experts. In *Proceedings of the 42nd International Conference on Machine Learning (ICML)*, PMLR, volume 267, pages 66772–66832.

[42] Samuel Williams, Andrew Waterman, and David Patterson. 2009. Roofline: An insightful visual performance model for multicore architectures. *Communications of the ACM*, 52(4):65–76.

[43] Thomas Wolf, Lysandre Debut, Victor Sanh, Julien Chaumond, Clement Delangue, Anthony Moi, Pierric Cistac, Tim Rault, Rémi Louf, Morgan Funtowicz, Joe Davison, Sam Shleifer, Patrick von Platen, Clara Ma, Yacine Jernite, Julien Plu, Canwen Xu, Teven Le Scao, Sylvain Gugger, Mariama Drame, Quentin Lhoest, and Alexander M. Rush. 2020. Transformers: State-of-the-art natural language processing. In *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing: System Demonstrations*, pages 38–45.

[44] An Yang, Anfeng Li, Baosong Yang, Beichen Zhang, Binyuan Hui, Bo Zheng, Bowen Yu, Chang Gao, Chengen Huang, Chenxu Lv, Chujie Zheng, Dayiheng Liu, Fan Zhou, Fei Huang, Feng Hu, Hao Ge, Haoran Wei, Huan Lin, Jialong Tang, Jian Yang, Jianhong Tu, Jianwei Zhang, Jianxin Yang, Jiaxi

Yang, Jing Zhou, Jingren Zhou, Junyang Lin, Kai Dang, Keqin Bao, Kexin Yang, Le Yu, Lianghao Deng, Mei Li, Mingfeng Xue, Mingze Li, Pei Zhang, Peng Wang, Qin Zhu, Rui Men, Ruize Gao, Shixuan Liu, Shuang Luo, Tianhao Li, Tianyi Tang, Wenbiao Yin, Xingzhang Ren, Xinyu Wang, Xinyu Zhang, Xuancheng Ren, Yang Fan, Yang Su, Yichang Zhang, Yinger Zhang, Yu Wan, Yuqiong Liu, Zekun Wang, Zeyu Cui, Zhenru Zhang, Zhipeng Zhou, and Zihan Qiu. 2025. Qwen3 technical report. *arXiv preprint arXiv:2505.09388*.

[45] Jenny Zhang, Shengran Hu, Cong Lu, Robert Tjarko Lange, and Jeff Clune. 2026. Darwin Gödel Machine: Open-ended evolution of self-improving agents. In *The Fourteenth International Conference on Learning Representations (ICLR)*.

[46] Yanli Zhao, Andrew Gu, Rohan Varma, Liang Luo, Chien-Chin Huang, Min Xu, Less Wright, Hamid Shojanazeri, Myle Ott, Sam Shleifer, Alban Desmaison, Can Balioglu, Pritam Damania, Bernard Nguyen, Geeta Chauhan, Yuchen Hao, Ajit Mathews, and Shen Li. 2023. PyTorch FSDP: Experiences on scaling fully sharded data parallel. *Proceedings of the VLDB Endowment*, 16(12):3848–3860.

[47] Lianmin Zheng, Zhuohan Li, Hao Zhang, Yonghao Zhuang, Zhifeng Chen, Yanping Huang, Yida Wang, Yuanzhong Xu, Danyang Zhuo, Eric P. Xing, Joseph E. Gonzalez, and Ion Stoica. 2022. Alpa: Automating inter- and intra-operator parallelism for distributed deep learning. In *Proceedings of the 16th USENIX Symposium on Operating Systems Design and Implementation (OSDI)*, pages 559–578.

A. Deterministic Five-Layer Verifier Specification

Table 8 provides the complete specification of the five-layer, 16-item deterministic verifier (§3.3–§3.4, Eq. (1)), comprising 11 always-on checks, 3 conditional checks, 1 informational check, and 1 promotion gate, alongside the per-item offline audit counts across 1,212 stored executions.

Table 8 | **Five-layer, 16-item deterministic verifier specification and offline audit across 1,212 stored executions** ("pri." denotes the execution's first failed layer; N_{tgt}: target updates; k: evaluation window; τ: relative tolerance, 0.03 by default).

Lay	Invariant ID	Mode	Implemented Predicate	Audit (1,212)
L1	`completes`	Always	Exit code 0, no timeout, and every requested update ran ($N_{\text{done}} = N_{\text{tgt}}$)	349 (349 pri.)
	`no_errors`	Always	No NaN/Inf loss, OOM, NCCL/CUDA error or traceback line (collective lines only after the last update of a clean exit are reported, not failed); loss and gradient norm not frozen across updates	153 (2 pri.)
	`memory_cap`	Always	Peak reserved device memory $\leq M_{\text{cap}}$	1 (1 pri.)
L2	`same_workload`	Always	Tokens (or samples) per update and the sequence length equal the reference at every update, and every declared static invariant (e.g. the global batch) is unchanged; unverifiable ⇒ unchecked, which rejects because the item is required	0 failed; 7 rejected as unverifiable
	`same_data`	Always	The per-update data digest equals the reference's (unchecked where the job logs none)	none
	`same_model`	Always	Model digest and trainable-parameter count equal the reference's; a candidate that stops reporting a key the reference reported fails	none
	`frozen_files`	Always	The hash of every file declared frozen is unchanged after the run	none
L3	`optimizer_steps`	Always	The optimizer-step counter equals the reference's at every update; with only a learning-rate proxy, an equal schedule is unchecked and a different one fails	none
L4	`loss_within_tolerance`	Always	Window-mean $\lvert\bar{\ell} - \bar{\ell}_{\text{ref}}\rvert / \lvert\bar{\ell}_{\text{ref}}\rvert \leq \tau$, with τ widened to $3\,\sigma_\ell\sqrt{2/k}$ (no cap) when the run stopped before the window's end or the reference's own repeat disagrees per update beyond τ; when the data order is verified (`same_data` passes) the largest per-update deviation is held instead	42 (41 pri.)
	`grad_norm_within_tol`	Always	First-update gradient norm within τ of the reference's; a report, not a veto, unless the same first batch is proven (`same_data` checked)	unchecked in every run
	`eval_matches`	Cond.	The configured evaluation of the resumed candidate within τ_{eval} of the reference's	active where eval configured
	`precision`	Info	The reported precision metadata compared with the reference's (informational)	reported only
L5	`timing_consistent`	Always	The candidate's gain on InfraRSI's own clock agrees with its gain in the code's reported step time, $\lvert\Delta \ln\rvert \leq 0.03$; a run without reliable clock timestamps fails when the reference had them	158 (0 pri.); all co-occur with crashed run
	`speed_reproduces`	Promo	Fresh back-to-back re-run (reference, candidate): relative window-mean speedup $T^{\text{re}}_{\text{ref}}/T^{\text{re}}_{\text{cand}} - 1$ exceeds the paired noise margin $\exp(2\sigma_{\text{pair}}) - 1$	promotion gate
	`resume_continues`	Cond.	A checkpoint saved at `save_at` and resumed continues the loss within τ	active where checkpoint exists
	`user_tests`	Cond.	Every configured test command exits 0	active where test commands exist
All	**16 invariants**	**5 layers**	**812 valid (67.0%); 400 rejected (33.0%): 352 execution, 41 numerical, 7 unverifiable**	**710 triggers**